

Sushant's Answers to the Take-Home Questionnaire

GreyOrange - Abacus Engineering Exercise

Candidate	Sushant
Submission date	August 2, 2026
Scope	Engineering Exercise only; Product Manager Exercise excluded
Status	SUBMISSION CANDIDATE - HOSTED URL PENDING

The tested code is frozen in a private GitHub repository. Grant reviewer access and complete the hosted Render deployment before recruiter submission.

Response coverage

1. Brand and Store Onboarding
2. Product Master Ingestion
3. Real-Time RFID Inventory Feed
4. Identity, Access, and User Management
5. Replenishment Policy Ingestion and Execution

Executive Summary

I designed and implemented a multi-tenant backend for onboarding Orange and its stores, ingesting its product master, processing RFID observations, controlling tenant- and store-scoped access, and creating explainable replenishment work.

The submitted vertical slice uses Python, FastAPI, PostgreSQL 17, SQLAlchemy, Alembic, and an independently scalable background worker. PostgreSQL is both the transactional system of record and the durable work inbox for the submitted deployment. Docker Compose provides a reproducible local environment, and a Render Blueprint defines the hosted web service, worker, and managed database.

The design prioritizes tenant isolation, safe retries, transactional consistency, effective-dated hardware and EPC mappings, retained RFID evidence, at-least-once processing with idempotent effects, and explainable replenishment decisions. The implementation addresses the required 100-store scenario. A regional Kafka architecture is a future scale path when measured traffic, replay requirements, or multiple consumers justify its operational cost.

Release candidate repository: <https://github.com/sushant5/greyorange-abacus-engineering-take-home> (private). The tested code is frozen under tag **submission-code-v1**; reviewer access and the hosted API URL still require account-level release steps before this document is sent to the recruiter.

Submitted Demo Scope

The reviewer-facing demo intentionally proves one complete business path:

- onboard Orange and all 100 stores through an idempotent bulk operation;
- create corporate and store-scoped users and enforce tenant/store authorization;
- import a representative product, SKU, UPC, and EPC catalog;
- accept simulated device-authenticated RFID batches through a durable worker;
- handle duplicate, conflicting, late, future-dated, and unmapped observations safely;
- query current inventory by store, zone, and SKU; and
- create a replenishment policy, evaluate it, and create and claim one deduplicated task; the complete guarded lifecycle is covered by integration tests.

Kafka/SQS, physical edge software, X.509/mTLS, enterprise OIDC/SCIM, object-storage imports, calibrated confidence scoring, book-inventory variance, regional cells, and a custom UI are production design options, not features claimed by the submitted demo.

High-Level Architecture

```
Platform operator -- X-Platform-Key --+
Orange users ----- Bearer JWT -----+--> FastAPI API --> PostgreSQL 17
Store readers ---- Device Key -----+
                                     |
                                     +-- durable jobs -+
                                     worker
```

The API is stateless and handles validation, authentication, authorization, transactional writes, and query endpoints. The worker leases durable jobs from PostgreSQL and performs catalog promotion, RFID state processing, and targeted replenishment recalculation. PostgreSQL holds canonical tenant, catalog, observation, inventory, identity, policy, task, and job state.

The hosted slice intentionally has few infrastructure dependencies. The database transaction that accepts RFID evidence also creates its durable work item, eliminating a dual-write gap. At larger measured scale, edge aggregation, partitioning, object storage, and regional Kafka cells are introduced incrementally.

1. Brand and Store Onboarding

Tenant Setup and Multi-Tenancy Model

I represent Orange as one tenant in a shared PostgreSQL schema. Tenant-owned records contain **tenant_id**; application queries and authorization are tenant-aware; and business identifiers use tenant-qualified uniqueness where appropriate, such as store codes within Orange.

I considered three tenancy models:

- Shared schema with **tenant_id**: operationally efficient, fast to provision, and suitable for transactional joins.
- Schema per tenant: stronger namespace separation, but more difficult migrations, analytics, and operations.
- Database per tenant: strongest physical separation, but substantially higher provisioning and operating cost.

I selected the shared-schema model for the submitted platform. Regulated or exceptionally large customers can later be placed in dedicated regional databases or cells. The implementation includes active tenant creation, tenant-scoped service paths, tenant-aware JWT authorization, and platform-only integration endpoints. Comprehensive composite tenant foreign keys and PostgreSQL Row Level Security are recommended defense-in-depth work and are not claimed as implemented.

Store Hierarchy and Configuration

The logical hierarchy is:

```
Orange tenant
+-- Organization unit: country / region / district / other
    +-- Store: stable code, name, timezone, status, configuration
        +-- SALES_FLOOR zone
        +-- BACKROOM zone
        +-- Optional RECEIVING / OTHER zones
```

Core identifiers and relationships are relational because they need integrity and indexing. Each zone keeps Orange's code/name separately from a normalized semantic kind used by inventory logic, so customer vocabulary can change without changing the calculation. Flexible store-specific settings are stored in JSONB so variations do not require a database column for every option; settings that the platform begins to execute should move into a typed, versioned configuration registry.

Each store has a stable code, name, valid IANA timezone, optional organization path, operational status, zones, flexible configuration, and registered readers. The request validator rejects duplicate store codes, invalid timezones, repeated or conflicting hierarchy units, duplicate zones, missing required sales-floor/backroom semantic roles, duplicate device serial numbers, and device references to unknown zones. That role-coverage check prevents activation of a store for which replenishment has no valid inputs.

Hardware-to-Store Mapping

RFID readers are tenant-owned devices in a registry. A reader is assigned to a store and zone through an effective-dated mapping with **effective_from** and **effective_to**.

Historical assignments are preferable to a mutable **store_id** because devices can move and observations can arrive late. A late event resolves against the server-owned mapping that was valid at its observation time; the device supplies identity and evidence, never its own store or zone. A PostgreSQL GiST exclusion constraint over the device and effective-time range prevents overlapping assignments, while a partial unique index permits only one open-ended current assignment.

Devices use API keys for RFID ingestion. The plaintext key is returned only when rotated, while its hash is stored. **antenna_port** and RSSI are retained with the observation. I did not invent a separate antenna topology because Orange's reader models, port behavior, and multiplexing rules were not provided; that model should be added after the hardware contract is known.

Environment Provisioning

Orange is provisioned logically within the shared Abacus SaaS environment; each physical store does not receive a separate application or database. The submitted environment contains a FastAPI web service, an independent worker, and PostgreSQL.

Docker Compose provides PostgreSQL, migrations, API, and worker services locally. The Render Blueprint places the API, worker, and managed PostgreSQL in one region, runs migrations before traffic, injects secrets through the hosting platform, and disables automatic deployment for the submitted release. Dedicated Orange infrastructure remains an option if data residency, regulation, or contractual isolation requires it.

Operational Workflow for 100 Stores

1. A trusted platform operator creates the Orange tenant.
2. A deterministic request is prepared for all 100 stores, including organization paths, timezones, zones, configuration, and readers.
3. The complete request is structurally validated before service execution.
4. The client calls the bulk onboarding endpoint with a stable **Idempotency-Key**.
5. Organization units, stores, zones, devices, assignments, and the onboarding result are created in one PostgreSQL transaction.
6. If anything fails, the transaction rolls back so Orange is not left with a partially provisioned estate.
7. An exact retry with the same key and payload returns the original result.
8. Reusing the key with different data returns a conflict instead of changing prior work.
9. Operators list the resulting stores and devices, rotate device API keys whose plaintext is shown once, and complete physical installation checks.

The API accepts 1-500 stores per request, so all 100 stores fit in one operation. The test suite contains an assignment-sized scenario with 100 stores and 200 representative readers. The demo activates stores after the metadata transaction because it has no physical installer integration. In production, long-lived store status remains separate from transient onboarding and commissioning attempts: metadata is accepted atomically, then each store progresses independently through resumable credential, heartbeat, walk-test, catalog, and zone-role gates. A derived readiness view exposes each blocking gate without overloading the store row. The platform workflow remains the same at 5,000 stores, but physical commissioning must become technician-driven and self-service.

Implemented Evidence

- **POST /v1/platform/tenants**
- **POST /v1/platform/tenants/{tenant_id}/stores:bulk-onboard**
- Store/device listing, credential rotation, device assignment, and assignment history endpoints
- Relational tenant, organization unit, store, zone, device, assignment, and onboarding batch models
- Deterministic 100-store request generator and PostgreSQL integration scenario

2. Product Master Ingestion

Proposed Ingestion Solution

I use a staged pipeline rather than writing Orange's source directly into the live catalog:

```
Upload --> Stage --> Validate and normalize --> Reconcile --> Promote
```

Orange uploads a UTF-8 CSV through an idempotent platform API and declares either **DELTA** or **FULL** mode. The platform records the source checksum, mode, filename, status, counts, and reconciliation evidence. Rows remain separate from canonical products until validation succeeds.

A **DELTA** import inserts or updates supplied records and never treats omission as deletion. A **FULL** import is an explicit complete snapshot and deactivates styles, SKUs, and active EPC bindings omitted from the file.

Validation and Transformation

Validation occurs at several levels:

- File: UTF-8 content, required headers, supported shape, 10 MB maximum size, and 100,000 data-row maximum.
- Field: required values, bounded identifiers, valid product-code characters, valid GTIN-8/12/13/14 check digits, and valid hexadecimal or EPC URI identifiers.
- Batch: conflicting style/SKU definitions, duplicate UPC ownership, duplicate EPCs, and EPC-to-SKU conflicts.
- Existing catalog: attempts to reuse another SKU's UPC or move an existing SKU to a different style are rejected.

Transformation is deterministic. Style/SKU codes are trimmed and uppercased, UPC separators are removed, EPCs are canonicalized, label whitespace is normalized, and optional attributes are parsed into JSON objects. Raw source values and normalized rows are both retained so errors remain explainable.

Reconciliation and Promotion

The staged import is compared with the current Orange catalog and produces a preview of inserted, updated, unchanged, and deactivated styles, SKUs, and EPC bindings. Each error records its row, field, rejected value, reason, and supporting evidence.

An import with any validation conflict is rejected without changing the active catalog. A valid import creates a durable job. The worker locks the import and tenant, promotes the complete import

transactionally, and persists actual reconciliation counts. Exact retries return the existing import; the same key with different content or mode returns a conflict.

Corrected catalog mappings can be followed by replay of quarantined observations. Raw RFID event evidence is retained while processing and resolution metadata can advance.

Storage Choice and Rationale

The canonical catalog uses PostgreSQL tables for product styles, SKUs, and effective-dated EPC bindings. Tenant-qualified relational constraints protect the identifiers and relationships on which inventory correctness depends. JSONB is limited to genuinely variable attributes.

PostgreSQL is preferable to a document database here because uniqueness, referential integrity, effective intervals, reconciliation, and atomic promotion are primary requirements. Effective-dated EPC mappings preserve history for late observations.

For substantially larger feeds, the original file should land in object storage and validation should run asynchronously in chunks using bulk loading such as PostgreSQL **COPY**. Orange's source schema, delivery mechanism, import frequency, authoritative deletion semantics, and correction process remain customer inputs.

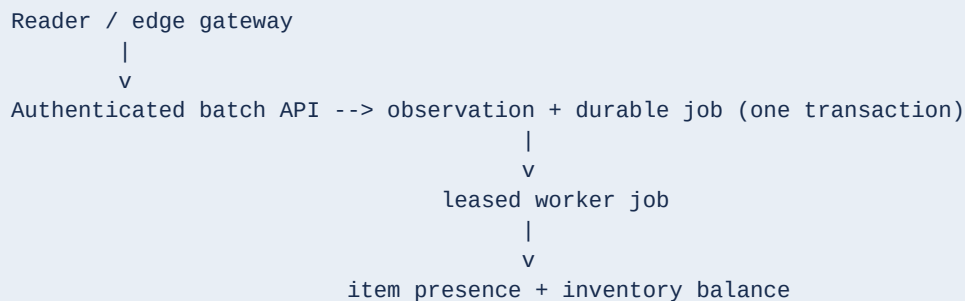
Implemented Evidence

- **POST /v1/tenants/{tenant_id}/catalog/imports**
- Import list/status and paginated row-error endpoints
- JWT-scoped SKU discovery endpoints
- Durable catalog-promotion worker job
- Staged raw/normalized rows, reconciliation preview, and transactional promotion

3. Real-Time RFID Inventory Feed

Ingestion and Processing Design

The production design places an edge gateway in each store to coalesce redundant reads, batch observations, buffer during outages, and retry with stable event identifiers. The submitted implementation begins at the authenticated HTTPS ingestion endpoint:



A reader submits a batch using its current rotated device API key. Each event contains a UUID, EPC, timezone-aware observation time, and optional reader sequence, antenna port, and RSSI. The API validates the complete envelope and authenticates both the device and active tenant.

The observation and its durable processing job are committed together. The API returns **202 Accepted** only after a successful commit. If the database is unavailable, the edge retains the batch and retries. Workers lease jobs using **FOR UPDATE SKIP LOCKED**, heartbeat while processing, retry with bounded backoff, and quarantine poison jobs after the configured attempt limit.

The guarantee is at-least-once processing with idempotent business effects. Exactly-once delivery is not claimed.

Event and Inventory Model

- **RfidObservation** retains event/batch identity, device, EPC, observation/ingestion times, resolved store/zone, antenna port, RSSI, payload hash, raw payload, and processing status.
- **InventoryItemState** stores the latest confirmed location for each tenant/EPC, candidate movement evidence, confidence, and last event time.
- **InventoryBalance** provides query-optimized quantities by tenant, store, zone, and SKU.
- **InventoryChange** records each confirmed transition for audit and targeted downstream processing.
- **DurableJob** provides restart-safe asynchronous work.

RFID observations are evidence, not automatically a sale, receipt, transfer, return, or shrink transaction. Derived observed presence is intentionally separate from a future book-inventory ledger.

Duplicate, Late, Unknown, and Noisy Reads

- Same event ID and same payload: return a duplicate disposition without another business effect.
- Same event ID and different payload: return an explicit conflict for investigation.
- Late event: retain it for audit, but do not allow it to regress newer current state.
- Excessive future timestamp: quarantine it so a bad clock cannot poison event-time state.
- Unknown EPC or missing effective device assignment: quarantine with a reason and permit replay after correction.
- Possible cross-zone movement: require configurable consecutive evidence within a bounded window.
- Missing read: never treat it alone as proof of absence.

Initial presence currently trusts one structurally valid, edge-filtered sighting. RSSI is retained, but universal thresholds are not assumed. Initial confirmation and signal rules must be tuned with measurements from Orange's actual readers, antennas, store layouts, and tagged merchandise.

Storage and Scaling Choices

PostgreSQL is used for the submitted slice because observations and durable jobs can be accepted atomically, while relational constraints and locks protect current state and aggregates. The stateless API and worker can scale independently.

The first scale steps are stronger edge filtering, larger batches, additional API/worker replicas, connection pooling, time/tenant partitioning, and archival of older raw evidence to object storage. At sustained multi-consumer volume or long replay windows, regional Kafka cells become appropriate. Presence streams would be keyed by tenant and EPC; inventory/replenishment streams would be re-keyed by tenant, store, and SKU.

SQS is a reasonable managed task-queue alternative, but Kafka better fits ordered partitions, long replay, and multiple independent streaming consumers. Neither Kafka nor SQS is part of the submitted implementation.

Implemented Evidence

- **POST /v1/device/read-batches**
- Paginated inventory query for tenant/store-scoped users
- Platform observation listing and quarantine replay
- Durable leased jobs with restart recovery
- Duplicate/conflict, event-time, quarantine, movement confirmation, and idempotent state tests

4. Identity, Access, and User Management

I separate authentication (who the principal is) from authorization (what that principal may do and over which stores).

User Creation and Authentication

The first Orange corporate administrator is created through a trusted deployment/bootstrap command. Authorized users can then create managers and associates through the REST API. A user belongs to one tenant, and normalized email addresses are unique within that tenant.

For reproducible reviewer testing, passwords are stored as Argon2id hashes. Login requires the Orange tenant code, email, and password and returns a short-lived JWT. The token contains identity and a version, but persisted grants remain the authorization truth. On each request the API validates the token, confirms the tenant/user are active, and reloads current roles and store grants. Suspending a user increments the token version and invalidates existing tokens.

Production should integrate Orange's OIDC/SSO provider, MFA, and optionally SCIM provisioning while retaining the same server-side resource authorization model.

Role and Store-Scope Model

- **CORPORATE_ADMIN**: tenant-wide administration, catalog/policy management, and cross-store inventory/replenishment access.
- **STORE_MANAGER**: inventory and replenishment operations over explicitly granted stores; management actions within those scopes.
- **STORE_ASSOCIATE**: inventory visibility and replenishment execution for explicitly granted stores.
- Device principal: authenticated RFID submission for its registered hardware identity only.

A user can hold multiple store grants. Protected endpoints verify the required operation, verified tenant, and persisted store scope. A URL/body tenant identifier is never accepted as authorization on its own. Cross-tenant and unassigned-store access is rejected without exposing another tenant's data.

The implementation retains tenant-scoped records for login attempts, user creation, and suspension. Invitations, password reset, grant-editing/resume workflows, richer corporate roles, distributed login throttling, and broader action auditing are production extensions.

Implemented REST APIs

- **POST /v1/auth/login** and **GET /v1/auth/me**
- **POST /v1/users**, paginated **GET /v1/users**, and **GET /v1/users/{user_id}**
- **POST /v1/users/{user_id}:suspend**

- **GET /v1/users/audit-records**
- Server-side tenant/store permission checks across inventory and replenishment routes

5. Replenishment Policy Ingestion and Execution

Policy Ingestion and Management

The implementation supports individual policy CRUD and an atomic JSON bulk-upsert of up to 1,000 policies. Each rule has a stable external key, optional store scope, selector (**SKU**, **STYLE**, **CATEGORY**, or **SIZE**), minimum/target/optional maximum floor quantities, priority, active flag, and timezone-aware effective interval.

Bulk ingestion requires an **Idempotency-Key**. An exact retry returns the original import; reusing the key with different content returns a conflict. Before changing canonical policies, the service validates quantity ordering, effective dates, store ownership, selector references, duplicate external keys, and ambiguous overlapping rules. If any row is invalid, no canonical policy is changed.

Policies are relational because store/catalog relationships, effective intervals, deterministic resolution, and indexed evaluation require strong constraints. Updates increment a revision counter; deletion is soft deactivation; and intervals are treated as **[effective_from, effective_to)**.

Deterministic Policy Selection

The winning active policy is resolved in this order:

1. It is active and effective at the evaluation timestamp.
2. A store-specific policy precedes a tenant default.
3. Selector specificity is **SKU > STYLE > CATEGORY > SIZE**.
4. Higher explicit priority wins.
5. An unresolved tie is rejected instead of choosing arbitrarily.

This avoids a general-purpose expression engine while keeping every decision deterministic and explainable.

Replenishment Calculation

```
if floor >= minimum:
    recommended = 0
else:
    need        = target - floor - existing_open_work
    available    = backroom - stock_reserved_for_open_work
    recommended = max(0, min(need, available))
```

The calculation reads confirmed RFID-derived balances in **SALES_FLOOR** and **BACKROOM** zones. Every evaluation stores the selected policy, thresholds, floor/backroom quantities, open-work reservation, inventory timestamp, formula, result, and reason. A zero result remains explainable: the floor is sufficient, existing work covers the shortage, the backroom has no available stock, or no policy matches.

Only one active task may exist for a tenant/store/SKU. Uniqueness constraints, row/advisory locks, and expected-version checks protect concurrent evaluation and task updates. Re-evaluation adds only uncovered work rather than duplicating assignments.

The submitted provisional lifecycle is:

```

OPEN --> CLAIMED --> IN_PROGRESS --> AWAITING_VERIFICATION --> VERIFIED
                                +-----> EXCEPTION
OPEN / CLAIMED -----+-----> CANCELLED (manager-authorized)

```

A claimed task is owned by its claimant. Moved quantity cannot decrease or exceed the requested amount, and verification requires the full amount to be recorded as moved. Confirmed RFID inventory changes enqueue targeted store/SKU recalculation. Policy changes use an explicit evaluation API in the submitted slice; automatic estate-wide policy fan-out is future work.

The formula, rule hierarchy, verification semantics, exception approvals, pooled style/category allocation, and SLAs are provisional because Orange did not provide those business rules. They require confirmation before production.

Implemented REST APIs

- Policy create/list/get/patch/deactivate under **/v1/tenants/{tenant_id}/replenishment/policies**
- **POST .../policies:bulk-upsert** and **GET .../policy-imports/{import_id}**
- **POST .../evaluations** and **GET .../evaluations/{run_id}**
- **GET .../tasks** and **PATCH .../tasks/{task_id}**

Testable REST APIs and Reviewer Walkthrough

Authentication Surfaces

Surface	Credential	Purpose
Platform integration	X-Platform-Key	Tenant/store/catalog setup and RFID remediation
RFID reader	X-Device-Key	Authenticated observation batches only
Orange user	Bearer JWT	User, catalog lookup, inventory, policy, evaluation, and task APIs

Representative API Inventory

Area	Representative endpoints
Operations	GET /health/live , GET /health/ready , GET /version , GET /docs , GET /openapi.json
Authentication	POST /v1/auth/login , GET /v1/auth/me
Users	POST/GET /v1/users , GET /v1/users/{id} , POST /v1/users/{id}:suspend , audit listing
Onboarding	POST /v1/platform/tenants , POST .../stores:bulk-onboard , store/device listing
Hardware	Device credential rotation, assignment creation, assignment history
Catalog	POST/GET .../catalog/imports , import status/errors, GET .../catalog/skus
RFID	POST /v1/device/read-batches , platform observation listing and replay
Inventory	GET /v1/tenants/{tenant_id}/inventory
Policies	Policy CRUD, POST .../policies:bulk-upsert , import status
Replenishment	POST/GET .../evaluations , GET/PATCH .../tasks

Reviewer Happy Path

1. Open **/health/ready**, **/version**, and **/docs**.
2. Log in as the Orange corporate reviewer and read the verified tenant identity from **/v1/auth/me**.
3. Use the platform integration credential to onboard the assignment-sized store batch.
4. Discover a reader, rotate its device API key whose plaintext is returned once, and retain the returned secret securely.
5. Create store-manager and associate accounts with explicit store grants.
6. Upload the sample product-master CSV and poll the import until completion.
7. Submit device-authenticated RFID observations and inspect inventory.
8. Import replenishment policies, run an evaluation, and exercise a generated task with scoped users.
9. Repeat representative requests to demonstrate idempotency and negative authorization.

Repository: <https://github.com/sushant5/greyorange-abacus-engineering-take-home> at tag **submission-code-v1**. Before submission, verify reviewer access and add the hosted base URL, Swagger/OpenAPI/version URLs, and dedicated reviewer credentials. Keep the platform integration key in the private recruiter email, never in the repository.

Deployment and Operations

The deployment Blueprint defines a paid Render web service, paid worker, and managed PostgreSQL 17 database in the same region. Paid instances are recommended for an immutable take-home link because sleeping services, worker unavailability, and database expiry create reviewer risk.

Migrations run before traffic. Secrets come from the hosting secret store. `/health/live` checks the process, `/health/ready` checks database connectivity and schema readiness, and `/version` identifies the release. Automatic deployment is disabled so the submitted commit remains fixed.

The intended release procedure is:

1. Create and review one Git commit/tag.
2. Run formatting, lint, strict type checking, PostgreSQL integration tests, coverage, migration drift checks, and the container build.
3. Apply the Render Blueprint and set the bootstrap administrator password.
4. Exercise the complete reviewer path and negative authorization tests against the hosted service.
5. Confirm `/version` matches the repository SHA/tag.
6. Insert the verified links and credentials into this document.
7. Disable automatic deployment and keep the paid services healthy through the review window.

Operational telemetry should cover request rate/error/latency, authentication denials, database CPU/I/O/connections, oldest durable-job age, retry/quarantine rate, RFID processing lag, catalog rejection rate, and replenishment backlog. Alerts should cover readiness failure, sustained job lag, repeated poison jobs, database capacity, and public endpoint availability.

Scale Path: 100 Stores to 5,000

Store count alone does not size an RFID system. Capacity depends on readers/antennas per store, post-edge-filter observations per second, burst and offline replay behavior, payload size, EPC cardinality, retention, consumer fan-out, and freshness objectives.

For the initial estate, retain the simple topology while measurements remain healthy: edge filtering/buffering, larger batches, independently scaled API/worker replicas, connection pooling, bounded concurrency, time/tenant partitioning, and object-storage archival.

For a materially larger estate, evolve to regional cells:



Kafka provides partitioned ordering, replay, and independent consumers. Regional cells limit blast radius and latency. This is a production evolution, not a claim that the submitted PostgreSQL inbox has been benchmarked for 5,000 stores.

Tradeoffs, Assumptions, Risks, and Known Limits

Key Tradeoffs

Selected approach	Alternative	Rationale
Shared schema with tenant scoping	Schema/database per tenant	Faster onboarding and simpler operations; dedicated cells remain possible for regulated customers
Atomic 100-store transaction	Partial per-store success	Deterministic correction/retry and no partially provisioned estate; asynchronous per-store stages suit much larger rollouts
Relational core plus selective JSONB	Document database	Protect critical identifiers and relationships while allowing variable attributes
Effective-dated mappings	Mutable device/EPC foreign keys	Preserves history for moved readers, catalog corrections, and late events
PostgreSQL durable jobs	Kafka/SQS from day one	Atomic acceptance and few hosted dependencies; broker added only at measured scale
Local Argon2id/JWT for review	Immediate enterprise SSO	Reproducible testing; production keeps authorization but replaces authentication with OIDC
Deterministic policy resolver	Generic rules engine	Easier explanation, validation, governance, and testing

Assumptions Requiring Orange Confirmation

- Shared SaaS is acceptable and no dedicated region/database is contractually required.
- Store codes, reader serial numbers, product identifiers, EPCs, and event UUIDs are stable external keys.
- The source product file is authoritative according to its declared **DELTA** or **FULL** mode.
- Store edge software can batch, buffer, and retry observations.
- Reader timestamps are timezone-aware and sufficiently synchronized for bounded skew handling.
- Sales-floor/backroom zones and the provisional replenishment formula reflect Orange's intended workflow.
- RFID presence is evidence, while POS/receiving/transfer/return/shrink systems provide book-inventory transactions.

Principal Risks and Mitigations

- Tenant leakage: server-derived tenant/store scopes, negative authorization tests, and future RLS/composite tenant keys.
- Noisy or missing reads: edge filtering, retained RSSI/evidence, movement hysteresis, freshness metadata, and no absence inference from one missing read.
- Late/duplicate events: stable event IDs, payload fingerprints, observation time, idempotent effects, and non-regressing current state.
- Bad product mappings: staged validation, row evidence, reconciliation, effective-dated bindings, quarantine, and replay.

- Reviewer availability: paid hosting, health/version endpoints, fixed commit, backups, and a Docker Compose fallback.
- Unmeasured scale: define workload/SLOs and publish load-test results before claiming throughput or latency.

Honest Implementation Boundaries

- The tagged source release is private until reviewer access is granted; no hosted URL is claimed before a verified deployment exists.
- The submitted worker polls PostgreSQL; Kafka and SQS are not implemented.
- Processing is at-least-once with idempotent effects, not exactly once.
- Comprehensive PostgreSQL RLS and composite tenant foreign keys are not implemented.
- Enterprise OIDC/SSO, SCIM, password reset, and MFA are production integrations.
- Store activation is immediate after atomic onboarding; external installation stages/checklists are future workflow.
- A separate antenna topology is not modeled without Orange's hardware contract.
- Formal book-inventory variance, receiving, transfers, returns, shrink, and POS integrations are not implemented.
- Automatic estate-wide policy fan-out and immutable before/after policy history remain production work.
- Platform store/device/assignment listing needs pagination before very large-estate operations.
- No 5,000-store throughput or near-real-time latency claim is made without a supplied workload and measured test.

Verification Evidence

The repository contains automated coverage for tenant/store onboarding, catalog validation and promotion, authenticated RFID ingestion, duplicate/conflict handling, late-event non-regression, quarantine/replay, store-scoped authorization, policy precedence, explainable calculation, task lifecycle, and concurrency controls. An assignment-sized test provisions 100 stores and 200 readers.

Local release verification on August 2, 2026 passed all 50 tests against PostgreSQL 17.10 with 82.38% total coverage, including branch measurement. Ruff lint/format, strict mypy, and Alembic migration drift checks also passed. The one-command reviewer runner completed the full path twice against a clean local API, worker, and database, including a safe idempotent rerun.

The code release is tagged **submission-code-v1**. Before recruiter submission, add the hosted URL and matching **/version** value after rerunning the same verifier against the deployed build.

Product Manager Exercise: Not applicable. This submission addresses the Engineering Exercise only.